MMO TCPFighter Server 만들기

MMO - Massively Multiplayer Online

C 스타일 / select 모델 / 단일 스레드

기본적인 싱글 스레드와 select 모델 방식이지만 이것조차 구현하지 못한다면 멀티 스레드 구조의 IOCP 서버는 절대 제대로 된 구현이 불가능 합니다. 필수적으로 완벽하-게 마무리 해야 합니다.

1. 가장 기본적인 소켓처리 방법으로 서버를 구현해봅니다.
2. select 모델을 쓰지만 64개의 소켓 이상을 처리 하도록 구현 해봅니다.
3. MMO 구현을 위해 구역별 섹터 관리를 구현합니다.

1. 서버의 메인 구조 설계
2. 로그 출력용 매크로 함수
3. MMO TCPFighter 서버 내부 구조
4. 네트워크 부 전역변수 . 구조체 . 함수
5. 컨텐츠 부 게임 로직의 처리
6. MMO 게임의 섹터 (구역) 처리
7. 섹터의 사이즈
8. 섹터의 구현
9. 클라이언트에게 메시지 보내기 함수들

1. 서버의 메인 구조 설계

```
// 게임 메인 루프를 위한 전역변수
// 메인 루프는 이 변수를 늘 검사하면서 서버가 꺼지는 순간을 감지 한다.

bool    g_bShutdown = false;



int _tmain(int argc, _TCHAR* argv[])
{
        timeBeginPeriod( 1 );           // 타이머 해상도 높이기

        LoadData();                     // 설정 및 게임데이터, DB 데이터 로딩
        netStartUp();                   // 네트워크 초기화, 리슨소켓 생성 및 listen

        // 서버는 listen 함수를 호출함과 동시에 실질적인 ON 상태 (접속자를 받는 상태) 가 된다.

        while ( !g_bShutdown )          // 서버 메인 루프, 전역의 g_bShutdown 값에 의해 종료결정
        {
                네트워크 처리 함수();           // 네트워크 송수신 처리

                // 업데이트는 게임의 로직 (월드,캐릭터, 몬스터, 이벤트 등…)
                // 실제 게임에서 돌아가야 하는 로직을 처리 한다.

                Update();               // 게임 로직 업데이트

                ServerControl();        // 키보드 입력을 통해서 서버를 제어할 경우 사용
                Monitor();              // 모니터링 정보를 표시,저장, 전송하는 경우 사용
        }

        // 서버의 종료 대기

        // 서버는 함부로 종료 해도 안된다.
        // DB 에 저장 할 데이터나 기타 마무리 할 일들이 모두 끝났는지 확인한 뒤에 꺼주어야 한다.


        netCleanUp();
        return 0;
}
```

일반 클라이언트 어플리케이션과 구조가 비슷합니다.
다만 네트워크 처리 함수와 제어관련 함수가 추가 됩니다.

void ServerControl(void)

```
{
        // 키보드 컨트롤 잠금,풀림 변수
        static bool bControlMode = false;

        //-------------------------------------------------------
        // L : 컨트롤 Lock  /  U : 컨트롤 Unlock  /  Q : 서버 종료
        //-------------------------------------------------------
        if ( _kbhit() )
        {
                WCHAR ControlKey = _getwch();

                // 키보드 제어 허용
                if ( L'u' == ControlKey || L'U' == ControlKey)
                {
                        bControlMode = true;

                        // 관련 키 도움말 출력.
                         wprintf(L"Control Mode : Press Q - Quit ₩n");
                        wprintf(L"Control Mode : Press L - Key Lock ₩n");
                }

                // 키보드 제어 잠금
                if ( (L'l' == ControlKey || L'L' == ControlKey) && bControlMode )
                {
                        wprintf(L"Control Lock..! Press U - Control Unlock₩n");

                        bControlMode = false;
                }

                // 키보드 제어 풀림 상태에서 특정 기능
                if ( (L'q' == ControlKey || L'Q' == ControlKey) && bControlMode )
                {
                        g_bShutdown = true;
                }

                // 기타 컨트롤 기능…
                if ( … ControlKey || … ControlKey) && bControlMode )
                {
                        // 기타 처리 하고자 하는 함수
                }
        }
}
```

일반적으로 서버의 제어는 외부 제어서버 및 관리툴과 네트워크 통신을 통해서 제어를 하지만 간혹 외부에서 네트워크 컨트롤이 안 되는 경우나 기능이 제공되지 않는 경우 서버콘솔에서 직접 해야 하는 경우가 생길 수 있습니다. 그럴 때 이 기능을 사용하게 될 것입니다. 다만 서비스 모드로 서버를 개발 한다면 불필요한 기능 입니다. 개발과정, 디버깅, 긴급상황 시 사용하는 목적 입니다.

로그 출력용 함수

서버에서 로그 출력은 아주 중요합니다. 콘솔 화면에 텍스트로 출력 하거나, 파일로 저장하거나, 외부 네트워크로 전송 또는 DB 에 저장 등 로그를 저장하는 방법도 다양 합니다. 개발단계에 필요한 로그, 테스트 단계에서 필요한 로그, 라이브 서비스에 필요한 로그 등… 상황에 따른 용도도 다릅니다. 개발 단계의 로그를 라이브 서비스에서 모두 남기면 속도의 저하와 불필요한 로그로 인하여 로그 데이터 전체가 의미 없어 지기도 합니다.

나중에는 로그 클래스를 만들어서 사용 할 것입니다만 첫 단계로 매크로를 사용해서 느낌만 살려 보겠습니다.
간단한 매크로와 전역 변수,함수를 사용해서 로그를 남기도록 해봅니다. Log 함수는 로그 레벨을 지정하여 지정된 레벨 이상의 로그만 출력 합니다.

```
int             g_iLogLevel;            // 출력.저장 대상의 로그 레벨
WCHAR           g_szLogBuff[1024];      // 로그 저장시 필요한 임시 버퍼
```

위 두 개의 변수는 전역으로 선언 하여 사용 합니다. g_iLogLevel 에 설정된 레벨 이상의 로그는 모두 출력됩니다.

```
#define dfLOG_LEVEL_DEBUG        0
#define dfLOG_LEVEL_ERROR        1
#define dfLOG_LEVEL_SYSTEM       2
```

로그 레벨은 보통 3단계로 나눌 수 있으며, 상황에 따라 그 이상 단계를 나누어도 됩니다. 로그를 남기고자 하는 순간 어떤 레벨에 해당되는 로그인지 지정하며, 현재 시스템 로그레벨 (g_iLogLevel) 이상인 경우에만 출력(저장) 됩니다.

```
void Log(WCHAR *szString, int iLogLevel)
{
        wprintf(L"%s ₩n);
}
```

단순히 입력된 문자열을 출력만 합니다. 실제로는 파일로 저장, DB 로 저장 등 기능들이 추가 되어야 합니다.
위 함수는 로그 문자열에 다양한 변수 사용이 어려우므로 매크로 함수의 가변인자를 사용하여 문자열 포멧 기능을 사용할 수 있도록 해봅니다. **(이는 로그 클래스를 만들어서 가변인자 함수로 변경해야 합니다. 지금은 연습 입니다)**

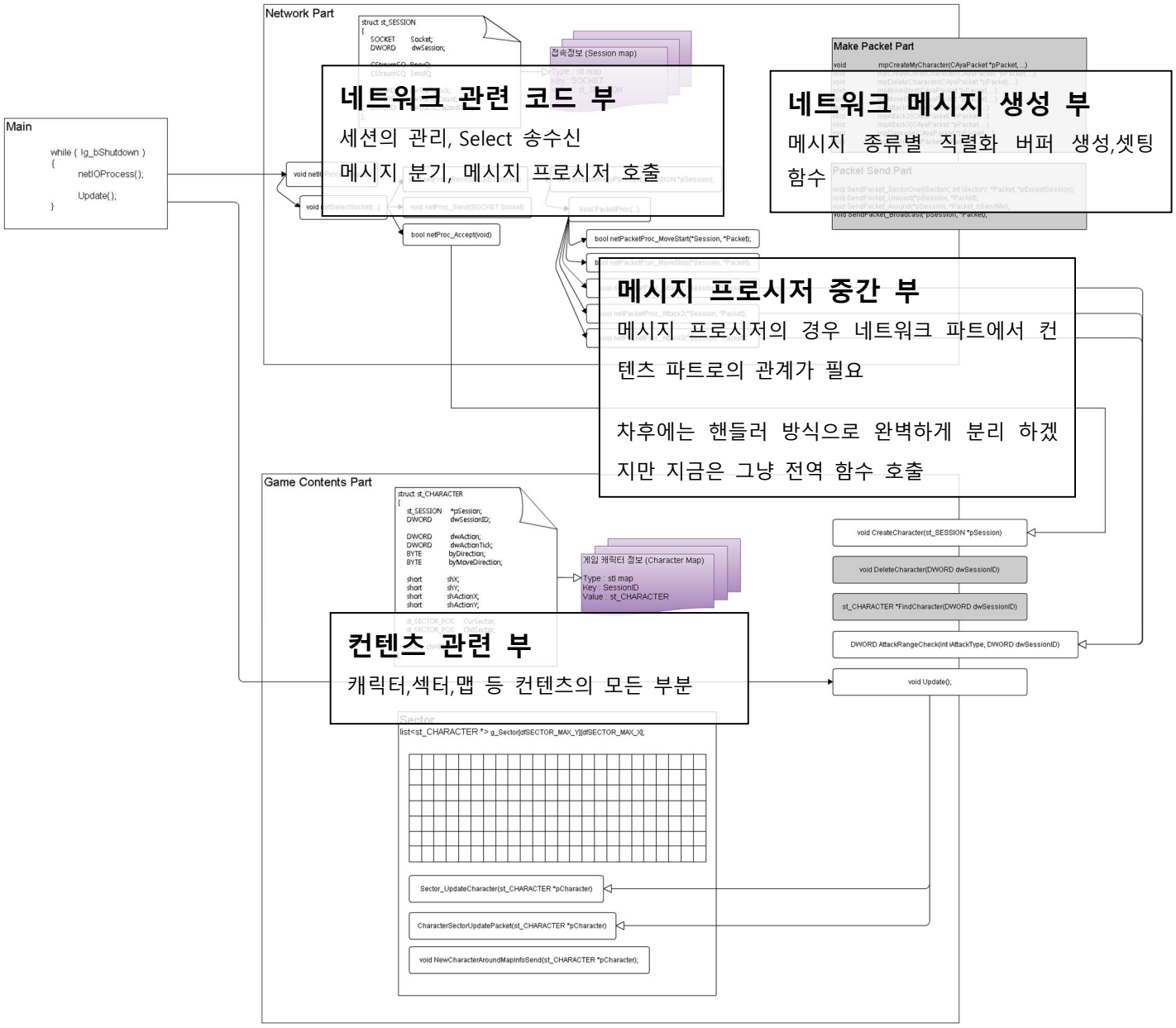
```
#define _LOG(LogLevel, fmt, ...)                                ₩
do {
                                                                ₩
        if ( g_iLogLevel <= LogLevel )                          ₩
        {                                                       ₩
                wsprintf(g_szLogBuff, fmt, ##__VA_ARGS__);      ₩
                Log(g_szLogBuff, LogLevel);                     ₩
        }                                                       ₩
} while (0)                                                     ₩
```

위 _LOG 매크로를 통해서 문자열의 변수대입과 로그레벨에 따른 처리가 가능해 졌습니다.
do while(0) 은 매크로 함수 사용시 if else 에서의 문제를 해결하기 위한 일반적인 방법 입니다.
매크로 내부 코드를 do ~ while(0) 로 묶지 않는다면

```
if ( 1 )
        _LOG(..);
else
        XXXXXXX
```

와 같은 코드에서 문법 오류가 발생 하므로 do ~ while(0) 을 사용하여 반복문은 아니지만 1회 실행 후 종료되는 매크로 블록을 만듭니다.

MMO TCPFighter 서버 내부 구조



전체적인 구조는 네트워크 파트와 컨텐츠 파트로 나눕니다.
네트워크 파트에서는 접속자 정보와 메시지 (패킷) 송수신을 담당하며, 컨텐츠 파트에서는 게임의 캐릭터 정보와 캐릭터 이동 및 대형 맵에 대한 구역별 메시지 전송을 위한 섹터를 관리 합니다.

위 그림에서 함수 하나하나는 의미가 없기 때문에 신경 쓰지 않으셔도 되며 기능과 기능에 대한 분리의 개념으로서 나누어서 만들어 보시면 되겠습니다

네트워크 부 전역변수 . 구조체

```
// 세션 정보 구조체.
struct st_SESSION
{
        SOCKET          Socket;                 // 현 접속의 TCP 소켓.
        DWORD           dwSessionID;            // 접속자의 고유 세션 ID.
        CRingQ          RecvQ;                  // 수신 큐.
        CRingQ          SendQ;                  // 송신 큐.
        DWORD           dwLastRecvTime;         // 메시지 수신 체크를 위한 시간 (타임아웃용)
};
```

위 구조체는 클라이언트 들의 세션 (접속정보) 구조체로 접속자 1명 당 하나씩 할당되는 구조체.
여기에는 게임의 컨텐츠는 들어가지 않는다. 오직 네트워크 송수신만을 담당 함.

```
SOCKET  g_ListenSocket          = INVALID_SOCKET;
map<SOCKET, st_SESSION *>       g_SessionMap;
```

- 접속자를 받기 위한 리슨소켓 / 세션 구조체를 관리하기 위한 map

Void 네트워크 처리 함수(void)

```
{
        전체 세션 맵을 돌면서 64개 단위로 끊어서 select 함수를 호출한다.

        '프로세싱 루프' 와 '세션 삭제 루프' 를 분리하여 연결 끊김 등의 세션 삭제 로직이 세션 순회 로직과
        꼬이지 않도록 해준다.

        for ( ... )
        {
                select > ReadSet 반응 > 수신&메시지처리함수();
        }
}
```

void 수신&메시지처리함수(SOCKET Socket)

```
{
        반응이 온 소켓을 대상으로 recv 후 RecvQ 를 대상으로 메시지 처리 함수 호출

        if ( !PacketProc(pSession, stPacketHeader.byType, &Packet) )
                return -1;

        return 0;
}
```

메시지 수신 시 처리 가능 한 메시지는 모두 처리 하는 것이 기본 입니다. 간혹 "한 클라이언트가 너무 많은 메시지를 보내서 클라이언트 한 명 처리에 묶여버리게 되면 어떻게 하느냐?" 라는 고민을 할 수 있습니다. 이는 서버에 부하가 걸릴 정도로 많은 메시지를 보내는 클라이언트가 문제이며, 일정량 (실제 클라이언트가 불가능한 수준) 이상의 메시지를 대량으로 보내올 경우는 공격이나 조작된 클라이언트로 판단하고 접속 종료나 기타 조치를 취해주어야 합니다.

프로토콜 설계상 어쩔 수 없이 한 클라이언트가 대량의 메시지를 보내야 하고, 이를 적절히 나누어서 처리 해주어야만 하는 게임이 만들어 졌다면 그때는 예외 상황으로 일정 수량의 메시지를 처리 후 다음 클라이언트로 넘어가게 끔 하는 편법을 쓸 수도 있습니다.

다만 이렇게 되는 경우는 Recv버퍼에 메시지가 계속 쌓이는 구조가 될 수 밖에 없으므로 애초에 잘못된 설계라고 볼 수 있습니다.

```
bool PacketProc(st_SESSION *pSession, BYTE byPacketType, CAyaPacket *pPacket)
{
        switch ( byPacketType )
        {
        case dfPACKET_CS_MOVE_START:
                return netPacketProc_MoveStart(pSession, pPacket);
                break;

        case dfPACKET_CS_MOVE_STOP:
                return netPacketProc_MoveStop(pSession, pPacket);
                break;

        case dfPACKET_CS_ATTACK1:
                return netPacketProc_Attack1(pSession, pPacket);
                break;

        case dfPACKET_CS_ATTACK2:
                return netPacketProc_Attack2(pSession, pPacket);
                break;

        case dfPACKET_CS_ATTACK3:
                return netPacketProc_Attack3(pSession, pPacket);
                break;

        }
        return TRUE;
}
```

실제 패킷 처리부분은 위와 같이 종류별로 나누어서 처리 합니다. 메시지의 종류가 수천 개 이상으로 너무 많은 경우는 종류별 (로비,던전,아이템 등..) 로 100~200개씩 단계를 거쳐서 switch 문으로 나누거나, 함수 포인터를 사용하기도 합니다.

switch 문은 case X 비교문이 정확하게 순서대로 숫자의 빠짐없이 배치가 된다면 비교분기가 아닌 점프 분기로 처리되지만, 비교 대상의 순서가 없거나 중간에 빠지는 숫자가 많은 경우 모든 내용을 분기로 처리하게 됩니다. 그런 경우는 자주 오는 메시지 순서로 정렬하여 상단에 배치 하는 것이 좋습니다.

위 패킷 프로시저에서는 netPacketProc_MoveStart, netPacketProc_MoveStop 두개만 살펴 보겠음.

```
bool netPacketProc_MoveStart(st_SESSION *pSession, CAyaPacket *pPacket)
{
	BYTE byDirection;
	short shX, shY;

	*pPacket >> byDirection;
	*pPacket >> shX;
	*pPacket >> shY;

	_LOG(dfLOG_LEVEL_DEBUG, L"# MOVESTART # SessionID:%d / Direction:%d / X:%d / Y:%d",
	                        pSession->dwSessionID, byDirection, shX, shY);

	//-------------------------------------------------------
	// ID 로 캐릭터를 검색한다.
	//-------------------------------------------------------
	st_CHARACTER *pCharacter = FindCharacter(pSession->dwSessionID);

	if ( pCharacter == NULL )
	{
		_LOG(dfLOG_LEVEL_ERROR, L"# MOVESTART > SessionID:%d Character Not Found!",
		                        pSession->dwSessionID);
		return false;
	}

	//-------------------------------------------------------
	// 서버의 위치와 받은 패킷의 위치값이 너무 큰 차이가 난다면 싱크 패킷을 보내어 좌표 보정.
	//
	// 본 게임의 좌표 동기화 구조가 단순한 키보드 조작 (클라이언트의 선처리, 서버의 후 반영) 방식으로
	// 클라이언트의 좌표를 그대로 믿는 방식을 택하고 있음.
	// 실제 온라인 게임이라면 클라이언트에서 목적지를 공유하는 방식을 택해야 함.
	// 지금은 좌표에 대해서는 간단한 구현을 목적으로 하고 있으므로
	// 서버는 클라이언트의 좌표를 그대로 믿지만, 서버와 너무 큰 차이가 있다면 강제좌표 동기화 하도록 함
	//-------------------------------------------------------
	if ( abs(pCharacter->shX - shX) > dfERROR_RANGE || abs(pCharacter->shY - shY) > dfERROR_RANGE )
	{
		mpSync(pPacket, pCharacter->dwSessionID, pCharacter->shX, pCharacter->shY);
		SendPacket_Around(pCharacter->pSession, pPacket, true);

		shX = pCharacter->shX;
		shY = pCharacter->shY;
	}

	//-------------------------------------------------------
	// 동작을 변경.  동작번호와, 방향값이 같다.
	//-------------------------------------------------------
	pCharacter->dwAction = byDirection;

	//-------------------------------------------------------
	// 단순 방향표시용 byDirection (LL,RR) 과
	// 이동시 8방향 (LL,LU,UU,RU,RR,RD,DD,LD) 용 MoveDirection 이 있음.
	//-------------------------------------------------------
	pCharacter->byMoveDirection = byDirection;
```

```
        //-----------------------------------------------------
        // 방향을 변경.
        //-----------------------------------------------------
        switch ( byDirection )
        {
        case dfPACKET_MOVE_DIR_RR:
        case dfPACKET_MOVE_DIR_RU:
        case dfPACKET_MOVE_DIR_RD:
                pCharacter->byDirection = dfPACKET_MOVE_DIR_RR;
                break;

        case dfPACKET_MOVE_DIR_LU:
        case dfPACKET_MOVE_DIR_LL:
        case dfPACKET_MOVE_DIR_LD:
                pCharacter->byDirection = dfPACKET_MOVE_DIR_LL;
                break;
        }
        pCharacter->shX = shX;
        pCharacter->shY = shY;



        // 정지를 하면서 좌표가 약간 조절된 경우섹터 업데이트를 함. (섹터는 차후 설명)
        //-------------------------------------------------------------------
        if ( Sector_UpdateCharacter(pCharacter) )
        {
                //-----------------------------------------------------------------
                // 섹터가 변경된 경우는 클라에게 관련 정보를 쏜다.(섹터는 차후 설명)
                //-----------------------------------------------------------------
                CharacterSectorUpdatePacket(pCharacter);
        }
        //-------------------------------------------------------------------

        mpMoveStart(pPacket, pSession->dwSessionID, byDirection, pCharacter->shX, pCharacter->shY);


        //-----------------------------------------------------
        // 현재 접속중인 사용자에게 모든 패킷을 뿌린다.(섹터 단위 패킷 전송 함수 )
        //-----------------------------------------------------
        SendPacket_Around(pSession, pPacket);
        return true;
}

void    mpMoveStart(CAyaPacket *pPacket, DWORD dwSessionID, BYTE byDir, short shX, short shY)
{
        st_PACKET_HEADER stPacketHeader;
        stPacketHeader.byCode = dfPACKET_CODE;
        stPacketHeader.bySize = 9;
        stPacketHeader.byType = dfPACKET_SC_MOVE_START;
        pPacket->Clear();
        pPacket->PutData((char *)&stPacketHeader, sizeof(st_PACKET_HEADER));
        *pPacket << dwSessionID;
        *pPacket << byDir;
        *pPacket << shX;
        *pPacket << shY;
}
```

컨텐츠 부 , 게임 로직의 처리

지금까지 설명된 부분은 네트워크 처리에 대한 부분이며, 게임서버 에서는 단순히 네트워크 송수신 외에도 실제 게임이 돌아가고 있어야 합니다. 단순한 채팅,메신저,웹, 기타 서비스 등의 대부분의 서버는 클라이언트의 요청 > 처리 > 응답 구조로 요청에 대한 처리만 해주면 되겠지만 게임서버는 접속한 수 천명의 클라이언트의 모든 플레이가 서버에서 실제로 구현이 되어야 합니다.

게임 서버의 모든 캐릭터는 클라이언트와 똑같이 움직이고 공격하고 동작 합니다. 그렇게 되어야만 중간에 다른 클라이언트가 접속을 하거나 섹터를 이동하는 과정에서 모든 클라이언트의 현 플레이 상황을 보여줄 수 있게 됩니다.

```
//-----------------------------------------------------------------
// 캐릭터 정보 구조체.
//-----------------------------------------------------------------
struct st_CHARACTER
{
        st_SESSION      *pSession;
        DWORD           dwSessionID;

        DWORD           dwAction;
        BYTE            byDirection;
        BYTE            byMoveDirection;

        short           shX;
        short           shY;

        st_SECTOR_POS   CurSector;              // 섹터 파트에서 설명
        st_SECTOR_POS   OldSector;

        char                    chHP;
};
```

캐릭터 객체는 위의 구조체를 사용하였으며, 고유 ID (세션), 액션정보, 좌표, 섹터정보 등을 가지고 있습니다. 위 구조체를 STL map 또는 unordered_map 으로 관리 합니다.

```
map<DWORD, st_CHARACTER *>   g_CharacterMap;
```

새로운 클라이언트가 접속 할 시 세션 객체 (st_SESSION)와 캐릭터 객체 (st_CHARACTER) 를 신규 할당하여 등록 합니다. 서버에서 필요한 캐릭터의 정보는 그래픽적인 요소를 제외하고, 컨텐츠를 모두 표현, 저장 , 처리 할 수 있는 내용을 가지고 있어야 합니다. mmo 게임은 서버에서 모든 컨텐츠 로직을 처리하기 때문에 결국 클라이언트의 캐릭터 구조(정보) 와 거의 흡사하게 구성 됩니다.

```
void Update(void)
{
        … 적절한 게임 업데이트 처리 타이밍 계산 …  // 25fps
                return;
        // 게임 업데이트 처리를 CPU 100% 을 먹도록 해줄 순 없을 것입니다.
        // 가장 좋은 것은 네트워크 송수신 스레드와, 로직 스레드를 분리하고
        // 로직 스레드의 루프는 클라이언트 프레임에 대응 되도록 맞추는 것입니다.
        // 지금 우리는 싱글 스레드로  네트워크 송수신 + 로직 루프 가 함께 처리 되어야 하는 상황이고
        // Sleep 같은 대기를 통해 로직 프레임을 제어하면 네트워크 송수신의 응답성이 너무 떨어지게
        // 됩니다. 그래서 기본 루프는 최대한 빠르게 돌리며 네트워크 처리를 시도하고, 로직 프레임은
        // 클라이언트 프레임과 비슷하도록 로직을 돌릴 의도로 설계 하였습니다.
        st_CHARACTER     *pCharacter = NULL;
        map<DWORD, st_CHARACTER *>::iterator Iter;
        for ( Iter = g_CharacterMap.begin(); Iter != g_CharacterMap.end(); )
        {
                pCharacter = Iter->second;
                Iter++;
                if ( 0 >= pCharacter->chHP )
                {
                        // 사망처리.
                        DisconnectSession(pCharacter->pSession->Socket);
                }
                else
                {
                        // 일정 시간동안 수신이 없으면 종료처리
                        if ( dwCurrentTick - pCharacter->pSession->dwLastRecvTime >
                                                dfNETWORK_PACKET_RECV_TIMEOUT )
                        {
                                DisconnectSession(pCharacter->pSession->Socket);
                                continue;
                        }

                        //------------------------------------------------------------
                        // 현재 동작에 따른 처리.
                        //------------------------------------------------------------
                        switch ( pCharacter->dwAction )
                        {
                        case dfACTION_MOVE_LL:
                                if ( CharacterMoveCheck(pCharacter->shX - dfSPEED_PLAYER_X,
                                                        pCharacter->shY) )
                                {
                                        pCharacter->shX -= dfSPEED_PLAYER_X;
                                }
                                break;

                        case dfACTION_MOVE_LU:
                                if ( CharacterMoveCheck(pCharacter->shX - dfSPEED_PLAYER_X,
                                                        pCharacter->shY - dfSPEED_PLAYER_Y) )
                                {
                                        pCharacter->shX -= dfSPEED_PLAYER_X;
                                        pCharacter->shY -= dfSPEED_PLAYER_Y;
                                }
                                break;
```

```
                case dfACTION_MOVE_UU:
                        if ( CharacterMoveCheck(pCharacter->shX,
                                                pCharacter->shY - dfSPEED_PLAYER_Y) )
                                                pCharacter->shY -= dfSPEED_PLAYER_Y;

                        break;
        case dfACTION_MOVE_RU:
                if ( CharacterMoveCheck(pCharacter->shX + dfSPEED_PLAYER_X,
                                        pCharacter->shY - dfSPEED_PLAYER_Y) )
                {
                        pCharacter->shX += dfSPEED_PLAYER_X;
                        pCharacter->shY -= dfSPEED_PLAYER_Y;
                }
                break;

        case dfACTION_MOVE_RR:
                if ( CharacterMoveCheck(pCharacter->shX + dfSPEED_PLAYER_X, pCharacter->shY) )
                {
                        pCharacter->shX += dfSPEED_PLAYER_X;
                }
                break;

        case dfACTION_MOVE_RD:
                if ( CharacterMoveCheck(pCharacter->shX + dfSPEED_PLAYER_X,
                                        pCharacter->shY + dfSPEED_PLAYER_Y) )
                {
                        pCharacter->shX += dfSPEED_PLAYER_X;
                        pCharacter->shY += dfSPEED_PLAYER_Y;
                }
                break;

        case dfACTION_MOVE_DD:
                if ( CharacterMoveCheck(pCharacter->shX, pCharacter->shY + dfSPEED_PLAYER_Y) )
                {
                        pCharacter->shY += dfSPEED_PLAYER_Y;
                }
                break;

        case dfACTION_MOVE_LD:
                if ( CharacterMoveCheck(pCharacter->shX - dfSPEED_PLAYER_X,
                                        pCharacter->shY + dfSPEED_PLAYER_Y) )
                {
                        pCharacter->shX -= dfSPEED_PLAYER_X;
                        pCharacter->shY += dfSPEED_PLAYER_Y;
                }
                break;
        }
        if ( pCharacter->dwAction >= dfACTION_MOVE_LL && pCharacter->dwAction <= dfACTION_MOVE_LD )
        {
                // 이동인 경우 섹터 업데이트를 함.
                if ( Sector_UpdateCharacter(pCharacter) )
                {
                        // 섹터가 변경된 경우는 클라에게 관련 정보를 쏜다.
                        CharacterSectorUpdatePacket(pCharacter);
                }
        }
    }
}
```

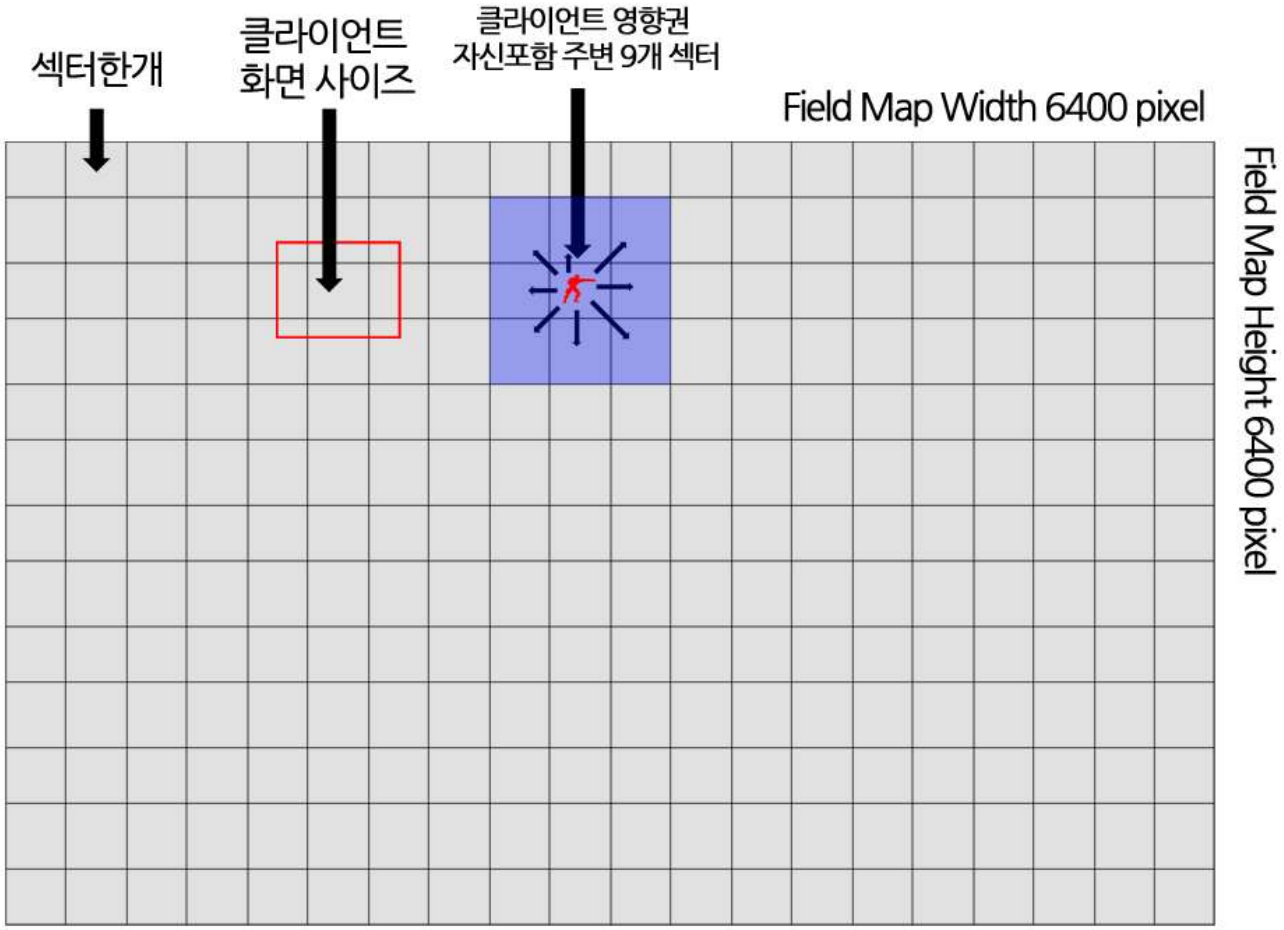
MMO 게임의 섹터 (구역) 처리

온라인 게임은 크게 규모를 기준으로 MMO (Mass Multiplay Online) 와 MO (Multiplay Onlien)로 나누어 집니다. MMOGAME 는 수 천명 이상의 유저가 같은 공간에서 함께 컨텐츠를 공유하며 플레이 하는 게임이며 MOGAME 은 5~20명의 유저가 방을 만들거나 독립된 채널을 통해서 소규모 인원으로 진행되는 게임 입니다.

MOGAME 에는 대표적으로 FPS 게임들, 레이싱 게임들, 액션게임이 있습니다. MMO 와 MO 가 복합되어서 로비나 필드에서는 MMO 로 플레이를 하다가 전투만 MO 로 진행되는 게임들도 있습니다.

MMO 게임들은 한 공간에 수 천명의 유저가 존재 할 수 있기 때문에 특정 캐릭터의 채팅, 이동, 액션, 변화의 정보를 모든 유저에게 보내주어서는 안됩니다. 예로 한 필드에 500명의 유저가 있을 때 한 캐릭터의 공격 메시지를 500명에게 모두 전송한다면 서버는 그 부하를 버틸 수 없으며 클라이언트 또한 500명의 캐릭터 정보를 모두 표현하고 로직을 처리 할 수 없습니다.

이를 위해서 커다란 필드 또는 맵을 구역별로 나누어서 (섹터 또는 셀,파티션 이라는 용어를 사용) 나누어서 해당 구역에 속한 유저들 간에만 변화와 행동 메시지를 보내 주어야 합니다.



일반적으로 MMO 게임은 픽셀좌표가 아닌 타일좌표를 사용하지만 우리가 만드는 TCPFighter 는 액션 게임 이므로 픽셀좌표를 사용 하고 있습니다. 타일 좌표이던 픽셀 좌표이던 캐릭터가 이동하는 간격만 다를 뿐 섹터 구현의 방법은 똑같습니다.

커다란 필드 맵을 일정 크기의 섹터 단위로 쪼개며 각각의 섹터에는 리스트 또는 배열 같은 자료구조가 들어갑니다. 캐릭터 (클라이언트) 는 자신이 서있는 좌표에 위치한 섹터에 들어가 있게 됩니다. 일반적으로 하나의 섹터는 캐릭터 객체의 포인터를 관리하는 리스트 또는 배열을 사용 할 수 있습니다.

캐릭터의 포인터가 섹터에서 관리 된다고 하여서 캐릭터 객체를 섹터에서 메인으로 관리 하는 것은 좋지 않습니다. 캐릭터 객체를 메인으로 관리하는 공간을 따로 존재하며, 섹터는 이의 서브적인 2차 인덱스 개념 입니다.

```
//-------------------------------------------------------------
// 캐릭터 객체 메인 관리
//-------------------------------------------------------------
map<DWORD, st_CHARACTER *>        g_CharacterMap;

//-------------------------------------------------------------
// 월드맵 캐릭터 섹터
//-------------------------------------------------------------
list<st_CHARACTER *>              g_Sector[dfSECTOR_MAX_Y][dfSECTOR_MAX_X];
```

코드로 보면 위와 같이 구현이 가능합니다. 캐릭터 객체를 메인으로 다루는 map 또는 list 와 캐릭터 객체 포인터를 다루는 리스트로 구성된 섹터의 2차원 배열 입니다. 개수는 전체 맵 사이즈와 섹터의 사이즈에 따라 결정 됩니다.

캐릭터는 이동을 할 때마다 자신의 좌표에 따라서 섹터와 섹터를 계속 이동하게 됩니다. 섹터와 섹터를 이동하는 과정에서는 이전 섹터에서 캐릭터 정보를 삭제하고, 새로운 섹터에 캐릭터 정보를 추가하는 과정이 반복 됩니다. 이런 부분에서 실수나 버그가 있을 경우 게임상에 복사 캐릭터가 나타나거나 영원히 삭제되지 않는 유령 캐릭터가 남기도 합니다.

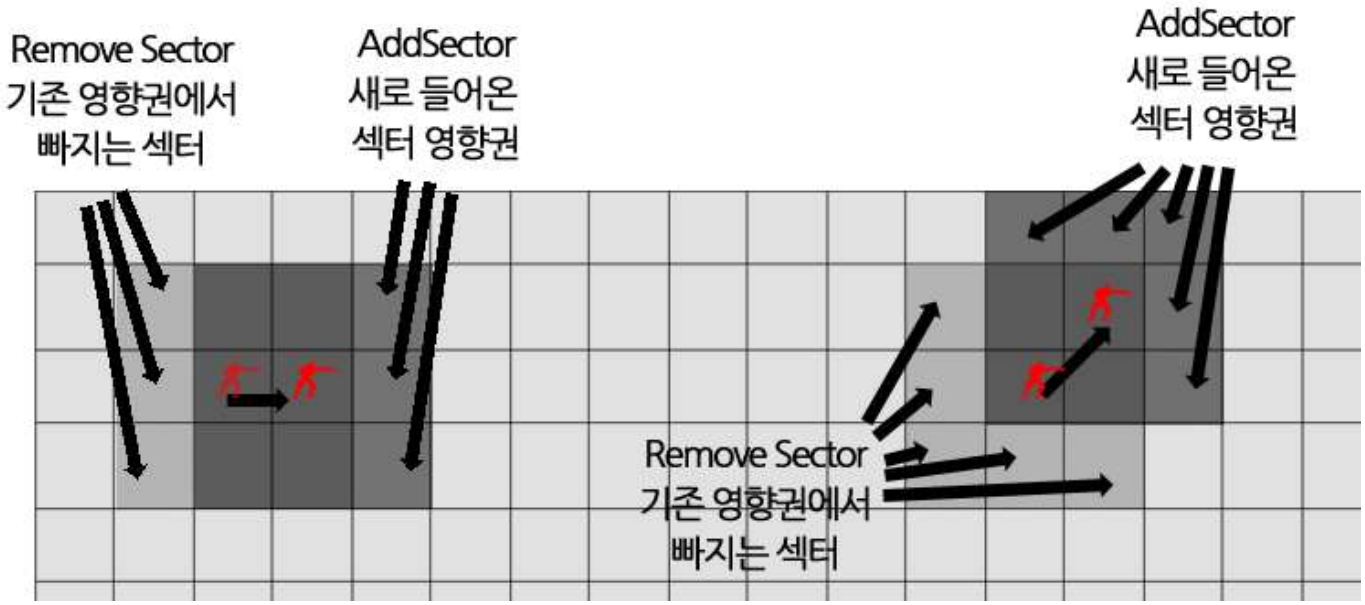
클라이언트는 영향권 섹터의 정보만

필드에 500명의 캐릭터가 있는 경우, 캐릭터들의 액션,이동 등 모든 메시지는 섹터 영향권 내에만 전달이 됩니다. 그리고 섹터 영향권 내의 정보만 알고 있어야 합니다. 나의 섹터 영향권에 위치하지 않은 캐릭터, 몬스터, 아이템 등등 어떤 정보도 알지 않아야 하며, 섹터를 벗어난 객체들은 클라이언트 에서도 삭제 되어야 합니다.

필드에 존재하는 500명의 캐릭터를 모든 클라이언트가 전부 로드하고 객체를 생성하여 관리한다면 이 또한 클라이언트의 부담이 엄청나게 됩니다.

그래서 서버는 모든 섹터를 관리 하면서 캐릭터가 섹터와 섹터를 이동하는 과정에서 객체들의 생성과 삭제 관련 메시지를 쉬지 않고 계속 보내주어야 합니다.

클라이언트는 매 순간순간 서버로부터 오는 객체 생성 메시지와 삭제 메시지를 처리하면서 새롭게 등장하는 캐릭터의 생성과 화면 영역 (섹터 영역) 을 벗어나서 삭제되는 캐릭터들의 삭제를 해주게 됩니다.



*현재 이동중인 캐릭터(클라이언트) 를 '나' 라고 표현

- RemoveSector - 영향권에서 벗어난 섹터. (화면에 보여지지 않게되는 섹터)

  나 에게 RemoveSector 에 위치한 캐릭터들의 삭제 정보를 보냄.
  RemoveSector 에 위치한 클라이언트들 에게 나의 삭제 정보를 보냄.

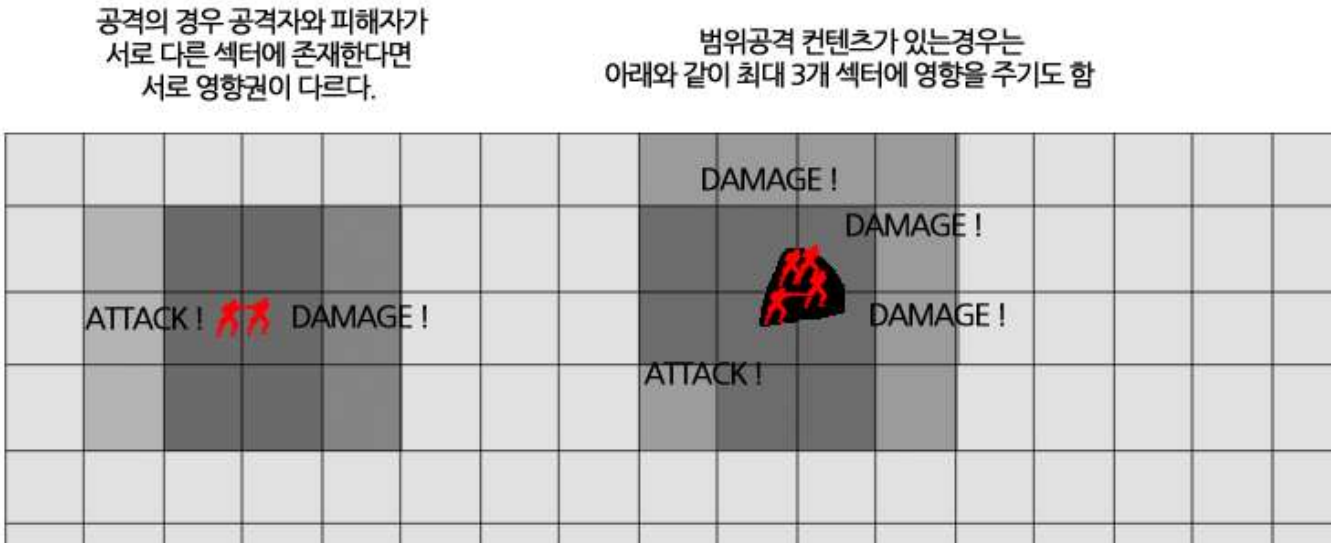
- AddSector - 새로 영향권에 들어온 섹터. (화면에 새로 나타나는 섹터)

  나 에게 AddSector 영역의 캐릭터,객체 생성 정보를 보냄. (생성 + 액션 포함)
  AddSector 에 위치한 클라이언트 들에게 나의 생성 정보를 보냄 (생성 + 액션 포함)

서버에서는 캐릭터가 이동하는 순간순간 마다 섹터의 정보를 갱신하고, 섹터간 이동이 발생하는 경우에는 위와 같은 처리가 필수적으로 필요 합니다.

객체 생성시 '생성 + 액션포함' 이라고 된 이유는 클라이언트에게 단순히 생성 메시지만 보낼 경우 해당 캐릭터가 생성만 됩니다. 만약 이동중인 캐릭터가 있는 경우는 생성과 동시에 이동메시지가 함께 보내져야지 정상적인 동기화가 맞추어 집니다.

섹터에 대한 정보 및 처리는 기본적으로 서버의 역할이므로 클라이언트는 이에 대해 크게 신경 쓸 부분이 없습니다. 클라이언트는 섹터에 대한 내용을 전혀 모르는 상태에서 서버의 캐릭터 생성,삭제 메시지만 제대로 처리를 해주어도 게임 플레이,동기화에 전혀 문제가 없습니다. 다만, 채팅 같은 경우 게임서버 외에 별도 채팅서버를 분리 시키는 경우가 있습니다. 이때는 클라이언트에서 자신의 섹터좌표를 알 수 있도록 하여 채팅서버에게 섹터좌표로 내 위치를 갱신 하는 경우도 있습니다.



이동 외에도 캐릭터가 타 캐릭터에게 영향을 주는 행위 (공격 등…) 이 있습니다.
같은 섹터 내에서 발생하는 경우는 아무런 문제가 없으나, 서로 다른 섹터에서 이런 행위가 발생 한다면 섹터 영향권이 다르므로 동기화 부분에서 문제가 발생 할 수 있습니다.

공격자 기준의 영향권에만 메시지를 보내게 되면 피해자 섹터 영향권에 메시지가 전달되지 않는 경우가 있을 수 있으며, 이렇게 되면 특정 클라이언트 에서는 캐릭터 HP 가 줄어들지 않는 등의 문제가 나타나게 됩니다.
피해자 기준으로 공격 메시지를 보내도 결국은 마찬가지 입니다.

이를 해결하기 위한 가장 간단한 방법은 공격 메시지와 데미지 메시지를 분리 시켜서 각 캐릭터에게 해당되는 메시지를 별도로 전달할 수 있습니다. 하지만 이는 게임 방식과 컨텐츠에 따라서 다르므로 상황에 맞게 적당한 방법으로 구현 해야 합니다.

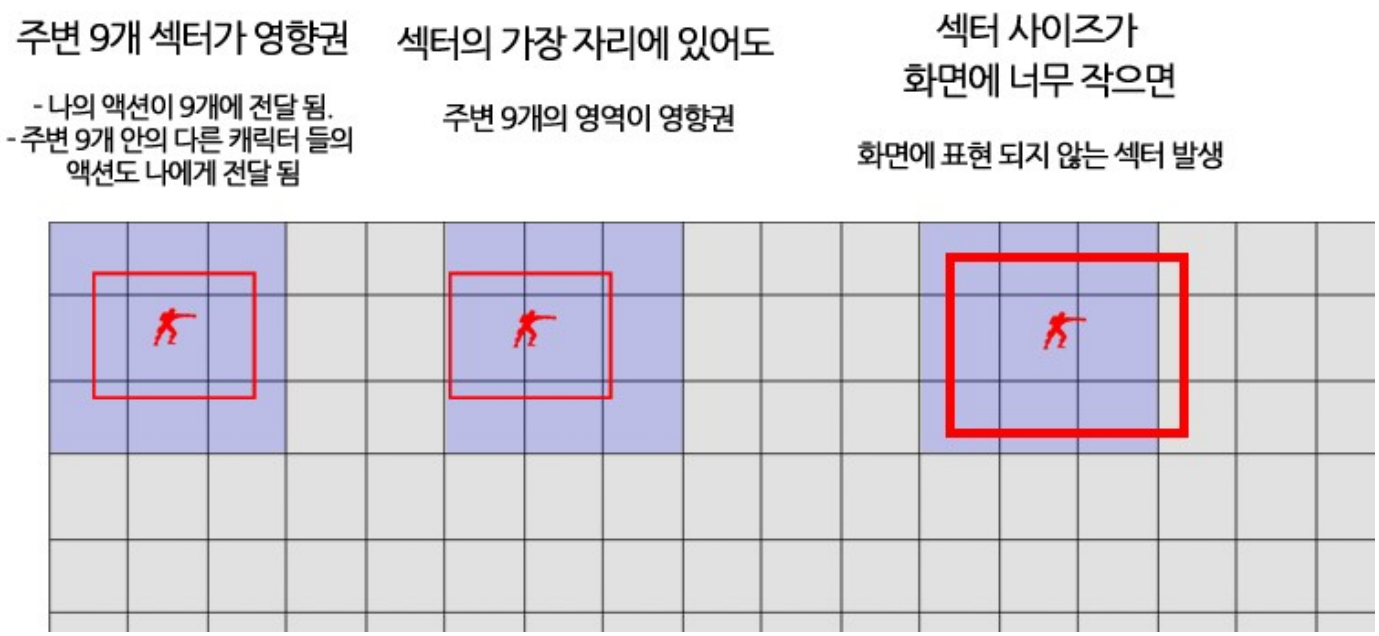
또한 클라이언트 입장에서도 특정 메시지에 대한 예외처리가 더 완벽하게 필요합니다.

공격 메시지를 받았는데 공격자 객체와 피해자 객체가 존재하지 않거나, 데미지 메시지를 받았는데 해당 객체가 없을 수도 있습니다. 모든 프로시저 함수에서는 객체 검색 후 검색에 실패한 경우를 확인 해서 진행해야 합니다.

섹터의 사이즈

섹터 사이즈는 화면의 크기와 캐릭터의 크기, 액션반경 등에 따라 결정 됩니다. 일반적으로 화면의 3/2 정도 크기가 무난 합니다.

섹터 크기를 결정하는 방법은 섹터의 가장자리에 캐릭터가 서 있는 경우에 자신 주변 9개의 섹터가 화면을 커버 할 수 있고, 캐릭터가 이동하는 과정에서 어색하게 생성되는 캐릭터가 없는 정도면 됩니다. 또는 섹터 사이즈에 따라서 섹터의 이동처리 및 네트워크 전송 량이 다르게 되므로 이는 게임의 특성에 맞게 테스트를 통해 적당한 사이즈를 찾아내야 합니다.



- 섹터 사이즈가 작을 경우
  - 캐릭터가 이동할 때 화면 가장 자리에서 갑자기 사라지거나, 생성되는 캐릭터가 보임
  - 캐릭터의 생성과 삭제가 빈번하게 일어난다.
  - 섹터간 이동 동기화 (캐릭터 생성,삭제) 처리가 자주 일어난다

  - 클라이언트가 관리 할 객체 개수가 적다.
  - 서버의 네트워크 메시지 전송 범위가 작다. (네트워크 부하가 적다)

- 섹터 사이즈가 클 경우
  - 클라이언트에서 관리 할 객체 수가 많아진다
  - 서버의 네트워크 메시지 전송 범위가 크다. (네트워크 부하가 크다)

  - 섹터 이동 횟수가 줄어든다. (섹터 이동에 따른 생성,삭제 처리 부하가 적어진다)

섹터 사이즈는 캐릭터의 크기, 화면의 크기, 클라이언트의 부하, 서버의 부하, 네트워크의 부하 등을 고려해서 조절해야 합니다.

섹터의 구현

```
//-----------------------------------------------
// 섹터 하나의 좌표 정보
//-----------------------------------------------
struct st_SECTOR_POS
{
        int     iX;
        int     iY;
};

//-----------------------------------------------
// 특정 위치 주변의 9개 섹터 정보
//-----------------------------------------------
struct st_SECTOR_AROUND
{
        int             iCount;
        st_SECTOR_POS   Around[9];
};
```

섹터 정보를 조금 더 편리하게 표현하기 위해서 위와 같은 구조체를 선언 하였습니다. 캐릭터 객체는 자신이 위치한 섹터의 정보 st_SECTOR_POS 를 가지고 있으며, 섹터의 이전 좌표와 현재 좌표 (Old, Cur) 두 가지 정보를 가지고 있도록 하였습니다. 섹터 이동시 이전 섹터와 현제 섹터를 비교하여 섹터의 이동을 구분하기 위함 입니다.
(꼭 old, cur 가 있어야만 되는 것은 아닙니다. 구현 방법에 따라서 다를 수 있습니다)

섹터 처리 함수 예시

섹터처리에 필요한 기능들을 함수로 만들어 보았습니다. 이는 예시 이므로 각자 알아서 해주시면 됩니다.

```
void    GetSectorAround(int iSectorX, int iSectorY, st_SECTOR_AROUND *pSectorAround);
```

- 특정 섹터 좌표 기준 주변 영향권 섹터 얻기.
- SectorX, SectorY 의 섹터좌표에서 주변 최대 9개의 섹터 좌표를 얻어 오는 함수

```
void    GetUpdateSectorAround( *pCharacter, st_SECTOR_AROUND *pRemoveSector, st_SECTOR_AROUND *pAddSector);
```

- 캐릭터가 섹터를 이동 한 뒤에 영향권에서 빠진 섹터, 새로 추가된 섹터의 정보 구하는 함수.

모든 캐릭터 이동 시에 (Update() , 메시지 처리, 등 어떤 사유로든 좌표가 변경되는 모든 순간) 에 섹터의 변경 여부를 확인하여 이에 대한 처리를 해주어야 합니다.

이 부분이 완벽하게 동작하지 않는 상태에서 추가적인 기능들을 넣는다면 원인 미상의 버그가 양산되므로 위 함수들의 정상동작 여부를 완벽하게 테스트 해야 합니다. 별도로 섹터 별 정보를 출력하는 테스트 프로그램 또는 로그 등을 남겨서 섹터간의 이동,삭제가 문제 없는지 확인 후 넘어갑니다.

클라이언트에게 메시지 보내기 함수들

섹터 개념이 없는 경우라면 메시지 보내기 함수는 유니캐스트와 브로드캐스트 두 가지면 해결이 가능합니다. 하지만 섹터에서는 조금 더 세분화 하여 용도에 맞게끔 사용 해야 합니다.

```
// 특정 섹터 1개에 있는 클라이언트들 에게 메시지 보내기
void SendPacket_SectorOne(int iSectorX, int iSectorY, CAyaPacket *pPacket, st_SESSION *pExceptSession);

// 특정 1명의 클라이언트 에게 메시지 보내기
void SendPacket_Unicast(st_SESSION *pSession, CAyaPacket *pPacket);

// 클라이언트 기준 주변 섹터에 메시지 보내기 (최대 9개 영역)
void SendPacket_Around(st_SESSION *pSession, CAyaPacket *pPacket, bool bSendMe = false);

// 진정 브로드 캐스팅 (시스템적인 메시지 외에는 사용하지 않음)
void SendPacket_Broadcast(st_SESSION *pSession, CAyaPacket *pPacket);
```

위 4가지 함수면 대부분 해결이 가능하며 게임 컨텐츠에 따라서 길드대상, 파티대상, 친구대상 등등 다양한 용도의 메시지 전송 함수들이 추가 될 수 있습니다.

현재 MMO TCPFighter 에서는 SendPacket_Broadcast 를 사용 하지 않았습니다.